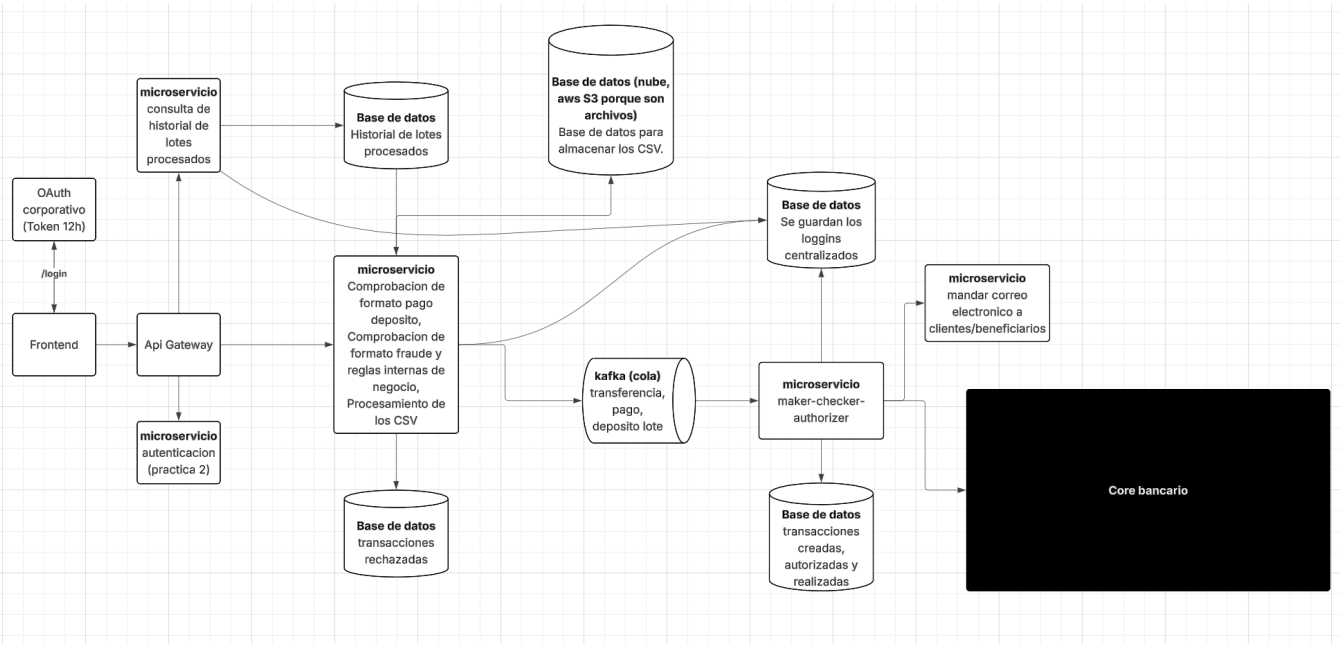
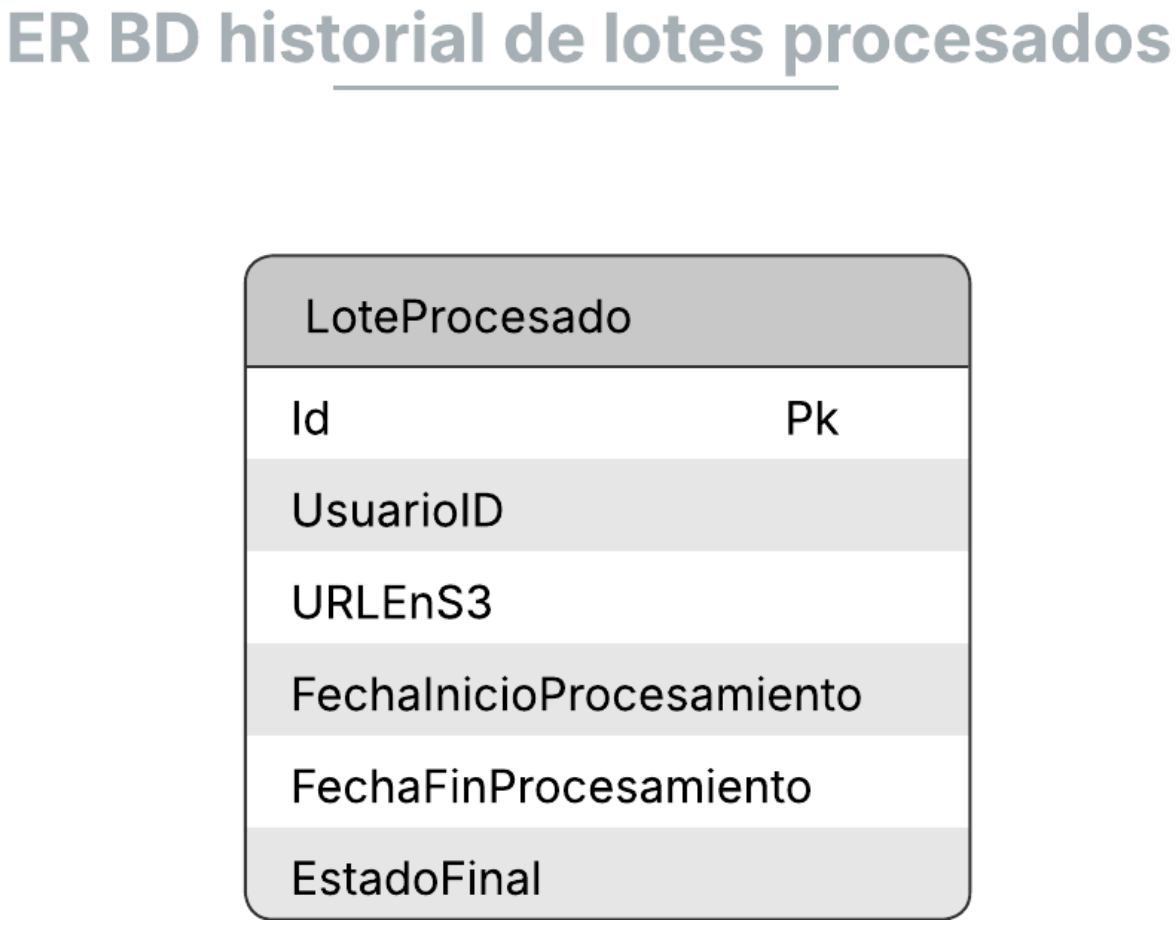

microservicio
consulta de historial de lotes procesados
Base de datos
Historial de lotes procesados
Base de datos (nube, aws S3 porque son archivos)
Base de datos para almacenar los CSV.
OAuth corporativo (Token 12h)
/login
Frontend
Api Gateway
microservicio
autenticacion (practica 2)
microservicio
Comprobacion de formato pago deposito, Comprobacion de formato fraude y reglas internas de negocio, Procesamiento de los CSV
Base de datos
transacciones rechazadas
Base de datos
Se guardan los loggins centralizados
kafka (cola)
transferencia, pago, deposito lote
microservicio
maker-checker-authorizer
microservicio
mandar correo electronico a clientes/beneficiarios
Base de datos
transacciones creadas, autorizadas y realizadas
Core bancario

ER de microservicios

Base de datos: Histiral de lotes procesados



esta al ser una simple tabla, recomiendo que sea NoSQL ya que en este caso al ser una simple tabla que no se puede normalizar va a funcionar de mejor manera, la estructura propuesta es la siguiente:

```
[
  {
    "_id": "lote_12345",
    "usuarioId": "user_789",
    "urlS3": "s3://bucket-banco/lotes/12345.csv",
    "tiempos": {
       "inicio": "2026-08-12T10:00:00Z",
       "fin": "2026-08-12T10:05:00Z"
    },
    "estadoFinal": "Procesado"
  },
  {
    "_id": "lote_12346",
    "usuarioId": "user_404",
    "urlS3": "s3://bucket-banco/lotes/12346.csv",
    "tiempos": {
       "inicio": "2026-08-12T11:15:00Z",
       "fin": "2026-08-12T11:15:30Z"
    },
    "estadoFinal": "Rechazado"
  }
]
```


Base de datos: Para almacenar los CSV

Se guardará en la nube, al ser archivos recomendaria utilizar S3 ya que esa se basa en guardado por archivos, si se necesita referenciar a algun CSV de estos, estará almacenada la ruta de acceso a estos en la base de datos anterior, su estructura es la siguiente:

Base de datos: Transacciones rechazadas

Base de datos de transacciones rechazadas

TransaccionesRechazadas	
Id	Pk
UsuarioID	
LoteID	
CuentaOrigen	
CuentaDestino	
Monto	
FechaOperacion	
MotivoRechazo	

```
[
  {
    "_id": "rech_98765a",
    "usuarioID": "usr_corp_042",
    "loteID": "lote_20260812_01",
    "cuentaOrigen": "3000123456",
    "cuentaDestino": "4000987654",
    "monto": 25000.00,
    "fechaOperacion": "2026-08-12T12:20:00Z",
    "motivoRechazo": "Saldo disponible insuficiente"
  },
  {
    "_id": "rech_98765b",
    "usuarioID": "usr_corp_042",
    "loteID": "lote_20260812_01",
    "cuentaOrigen": "3000123456",
    "cuentaDestino": "0000000000",
    "monto": 150.50,
    "fechaOperacion": "2026-08-12T12:20:01Z",
    "motivoRechazo": "Regla de prevención de fraude: cuenta de destino no válida"
  }
]
```


Base de datos: Se guardan los loggins centralizados

Para este caso, donde existe la necesidad de filtrar registros por múltiples dimensiones (tipo de log, fecha, microservicio y cuentas involucradas), se propone una estrategia de almacenamiento híbrida.

Para el almacenamiento en caliente (logs menores a 30 días), se utilizará una base de datos NoSQL orientada a documentos para optimizar la escritura. Para el almacenamiento histórico (logs mayores a 30 días), se optará por una base de datos columnar como BigQuery, lo cual permite trasladar los datos a un nivel más económico y aprovechar su estructura para consultas eficientes.

Base de datos loggins centralizados

Bitacora	
Id	Pk
FechaHora	
MicroservicioOrigen	
NivelLog	
Mensaje	
DetallesContexto	

Base de datos: Transacciones creadas, autorizadas y realizadas

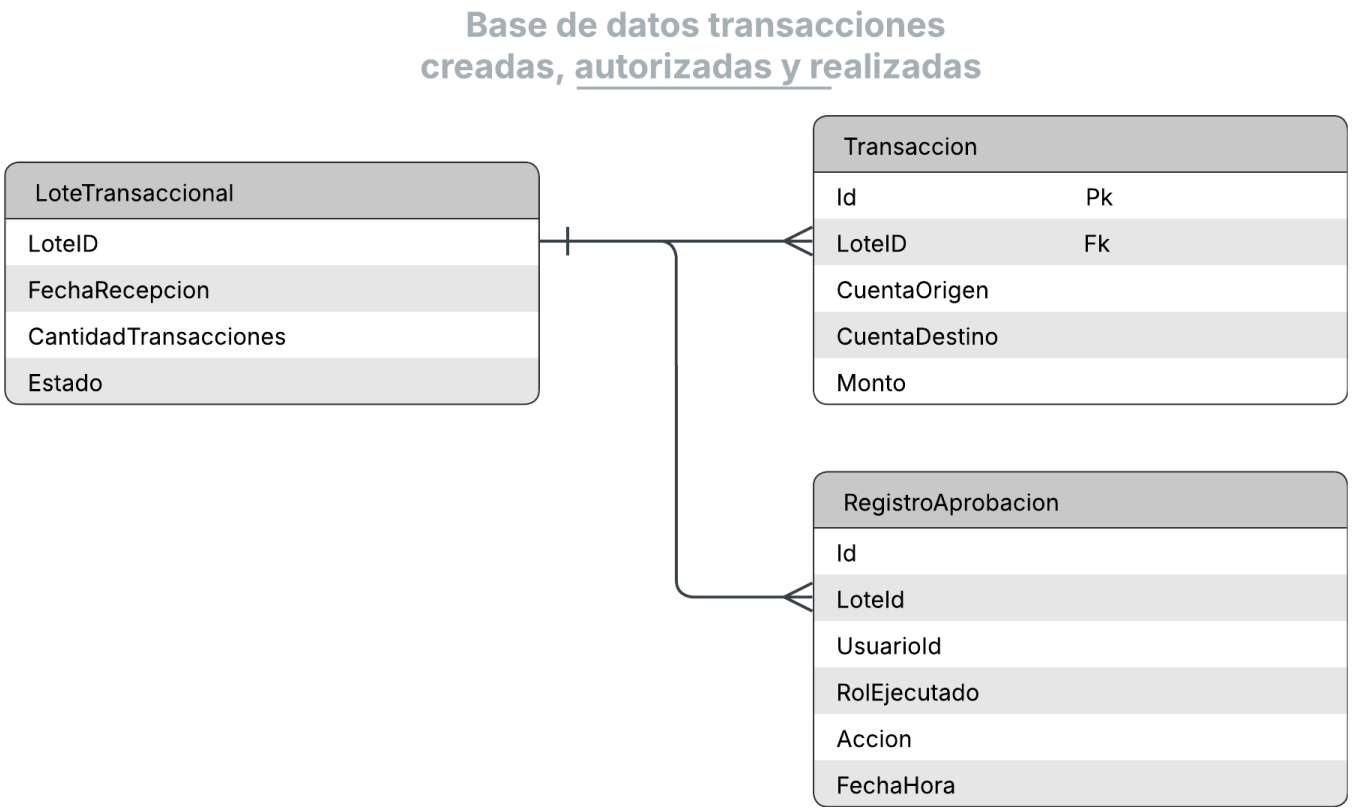


Diagrama de Clases UML (Microservicio Core Transaccional)

¿Qué representa?
Es la estructura interna orientada a objetos de tu microservicio principal (Maker-Checker-Authorizer). Refleja cómo se estructuran los datos en la memoria y en la base de datos relacional.

¿Por qué se diseñó así? (Justificación)

Diseño Orientado al Dominio (DDD): Se definió LoteTransaccional como la "Entidad Raíz". En la industria bancaria, las aprobaciones corporativas no se hacen transferencia por transferencia, sino por archivos masivos. Cambiar el estado de la cabecera actualiza lógicamente miles de transacciones de un solo golpe.

Separación de Auditoría: La clase RegistroAprobacion es la clave para cumplir con las reglas del negocio. Al separar la bitácora de la tabla de transacciones puras, garantizamos que el sistema pueda verificar si el usuario que intenta actuar como Checker es diferente al que actuó como Maker, creando un historial inalterable para auditorías financieras.

classDiagram namespace Microservicio_Maker_Checker_Authorizer { class LoteTransaccional { +String Id +Date FechaRecepcion +Int CantidadTransacciones +Float MontoTotal +String Estado +cambiarEstado(nuevoEstado) } class Transaccion { +String Id +String LoteId +String CuentaOrigen +String CuentaDestino +Float Monto } class RegistroAprobacion { +String Id +String LoteId +String UsuarioId +String RolEjecutado +String Accion +Date FechaHora +registrarAuditoria() } } LoteTransaccional "1" *-- "*" Transaccion : contiene LoteTransaccional "1" *-- "*" RegistroAprobacion : posee bitácora de

Diagrama de Secuencia UML (Flujo Crítico de 3 Pasos)

¿Qué representa?
Es la vista cronológica del sistema. Muestra exactamente en qué orden se comunican los componentes a través del tiempo cuando un usuario corporativo intenta aprobar un lote.

¿Por qué se diseñó así? (Justificación)

Seguridad Centralizada: Se colocó al API Gateway y al MS de Autenticación como los primeros interceptores. El core transaccional nunca confía ciegamente en una petición web; el Gateway siempre valida el token primero.

Manejo de Estados: Demuestra cómo la misma ruta de API (/aprobar) reacciona distinto dependiendo del estado actual del lote, avanzando secuencialmente de Pendiente_Checker a Lote_Completado.

Desacoplamiento Asíncrono (Kafka): Nota cómo, al final del paso 3, el servicio no espera a que se envíen los correos. En su lugar, publica un evento en Kafka y le

devuelve un 200 OK rápido al usuario. Esto evita que el frontend se quede "congelado" esperando confirmaciones de servidores de correo externos, delegando esa carga al microservicio de Notificaciones en segundo plano.

sequenceDiagram autonumber participant F as Frontend participant AG as API Gateway participant Auth as MS Autenticación participant MS as MS Core Transaccional participant K as Kafka (Eventos) participant CB as Core Bancario Externo participant MN as MS Notificaciones Note over F, MS: Paso 1: Ejecución del Maker F->>AG: POST /lotes/{id}/aprobar (Rol: Maker) AG->>Auth: Validar Token OAuth Auth-->>AG: Token Válido AG->>MS: Enrutar petición MS->>MS: Insertar RegistroAprobacion (Maker) MS-->>F: 200 OK - Estado: Pendiente_Checker Note over F, MS: Paso 2: Ejecución del Checker F->>AG: POST /lotes/{id}/aprobar (Rol: Checker) AG->>MS: Enrutar petición (Validación interna) MS->>MS: Insertar RegistroAprobacion (Checker) MS-->>F: 200 OK - Estado: Pendiente_Authorizer Note over F, CB: Paso 3: Ejecución del Authorizer y Envío F->>AG: POST /lotes/{id}/aprobar (Rol: Authorizer) AG->>MS: Enrutar petición MS->>MS: Insertar RegistroAprobacion (Authorizer) MS->>CB: POST /transferencias (Payload Lote Autorizado) CB-->>MS: 201 Created (Transacciones procesadas) MS->>K: Publicar Evento: "Lote_Realizado_Exitosamente" MS-->>F: 200 OK - Estado: Lote_Completado Note over K, MN: Proceso Asíncrono Desacoplado K-->>MN: Consumir Evento "Lote_Realizado_Exitosamente" MN->>MN: Renderizar plantilla de correo MN->>F: Enviar correo electrónico a clientes

Diagrama de Componentes UML

¿Qué representa?
Es la vista de "pájaro" o el mapa macro de toda tu infraestructura. Muestra las piezas de software desplegadas y cómo se comunican por la red.

¿Por qué se diseñó así? (Justificación)

Persistencia Políglota: El diagrama justifica visualmente por qué usamos diferentes bases de datos según el requerimiento. Se utiliza un Bucket S3 para archivos pesados, PostgreSQL para transacciones ACID exactas, MongoDB para un historial de documentos flexible y BigQuery para soportar analítica columnar sobre millones

de logs.

Arquitectura Orientada a Eventos: Al colocar a Kafka en el centro de las operaciones de éxito, rompemos las dependencias directas entre microservicios (evitando el acoplamiento HTTP). Si el servicio de Notificaciones se cae temporalmente, el Core Transaccional sigue funcionando y aprobando lotes sin enterarse; Kafka simplemente guardará los mensajes hasta que el servicio de correos vuelva a estar en línea.

Observabilidad Transversal: Las líneas punteadas hacia la base de datos de Logging demuestran que el monitoreo es una responsabilidad compartida por todos los microservicios, cumpliendo con el requerimiento de auditoría centralizada.

graph TD %% Definición de Nodos UI[Frontend Web] AG[API Gateway] subgraph Capa de Microservicios MAuth[MS Autenticación Práctica 2] MProc[MS Procesamiento CSV] MHist[MS Historial] MCore[MS Maker-Checker-Authorizer] MNotif[MS Notificaciones] end subgraph Persistencia y Almacenamiento S3[(AWS S3 - CSVs)] DBRech[(DB Rechazos - PostgreSQL)] DBHist[(DB Historial - MongoDB)] DBCore[(DB Transaccional - PostgreSQL)] DBLogs[(DB Logging - BigQuery)] end subgraph Bus de Mensajería Kafka((Apache Kafka)) end subgraph Sistemas Externos OAuth[OAuth Corporativo] CoreBank[Core Bancario Externo] end %% Relaciones de Flujo UI -->|Peticiones HTTP| AG %% Gateway a MS AG --> MAuth AG --> MProc AG --> MHist AG --> MCore %% Conexiones MS a Externos/DBs MAuth -->|/login| OAuth MProc -->|Guarda original| S3 MProc -->|Guarda errores| DBRech MProc -->|Publica tx válidas| Kafka MCore -->|Consume tx válidas| Kafka MCore -->|Guarda estados/auditoría| DBCore MCore -->|Envía lote aprobado| CoreBank MCore -->|Publica evento lote finalizado| Kafka MHist -->|Lee/Escribe registros| DBHist MNotif -->|Consume eventos| Kafka %% Patrón de Logging Centralizado MProc -.->|Envía logs| DBLogs MAuth -.->|Envía logs| DBLogs MCore -.->|Envía logs| DBLogs MHist -.->|Envía logs| DBLogs %% Estilos básicos classDef database fill:#f9f,stroke:#333,stroke-width:2px; class S3,DBRech,DBHist,DBCore,DBLogs database;